Your Quick Reference to

# KUBERNETES CHEAT SHEET

Keep this handy. Use it daily!

## Essential kubectl Commands Every DevOps Engineer Uses

### CLUSTER

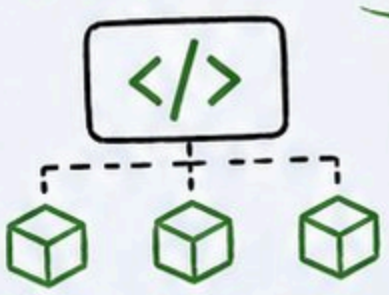
Your Kubernetes infrastructure

```
$ kubectl
$ kubectl get all --all-namespaces
NAMESPACE     NAME                       READY   STATUS    RESTARTS   AGE
kube-system   coredns-5d78c9869d-xxx     1/1     Running   0          2d
kube-system   metrics-server-7f6bxxx     1/1     Running   0          2d
default       nginx-deploy-6f7dxxx       1/1     Running   0          1d
$
```

### PODS

Where your containers run

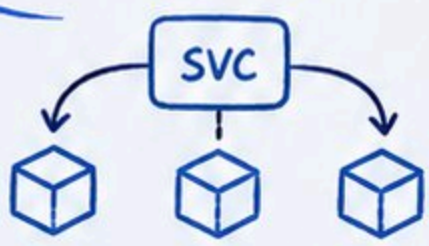
### DEPLOYMENTS




Manage replicas and updates

One Cluster. Infinite Possibilities.

### SERVICES




Expose and connect your apps

Save This Before Your Next Kubernetes Debugging Session!

KUBERNETES CHEAT SHEET

GET ORIENTED BEFORE YOU DEPLOY!

# 2 CLUSTER & CONTEXT

Essential commands to manage your cluster and switch between environments.

Your control plane for everything

## 1 CLUSTER INFO

```
$ kubectl cluster-info
Kubernetes control plane is running at https://1.2.3.4:6443
CoreDNS is running at https://1.2.3.4:6443/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy
```

- Displays cluster endpoint and core services.
- Helps verify connectivity to the cluster.

## 2 GET ALL CONTEXTS

```
$ kubectl config get-contexts
CURRENT   NAME      CLUSTER        AUTHINFO    NAMESPACE
*         dev       dev-cluster    dev-user    dev
          staging   stage-cluster  stage-user  staging
          prod      prod-cluster   prod-user   prod
```

- Lists all available contexts.
- The * shows your current context.

## 3 SWITCH CONTEXT

```
$ kubectl config use-context <name>
# Example
$ kubectl config use-context prod
Switched to context "prod".
```

- Switch to another environment.
- Always confirm before making changes!

## 4 LIST NODES

```
$ kubectl get nodes
NAME      STATUS   ROLES          AGE   VERSION
node-01   Ready    control-plane  12d   v1.29.2
node-02   Ready    <none>         12d   v1.29.2
node-03   Ready    <none>         12d   v1.29.2
```

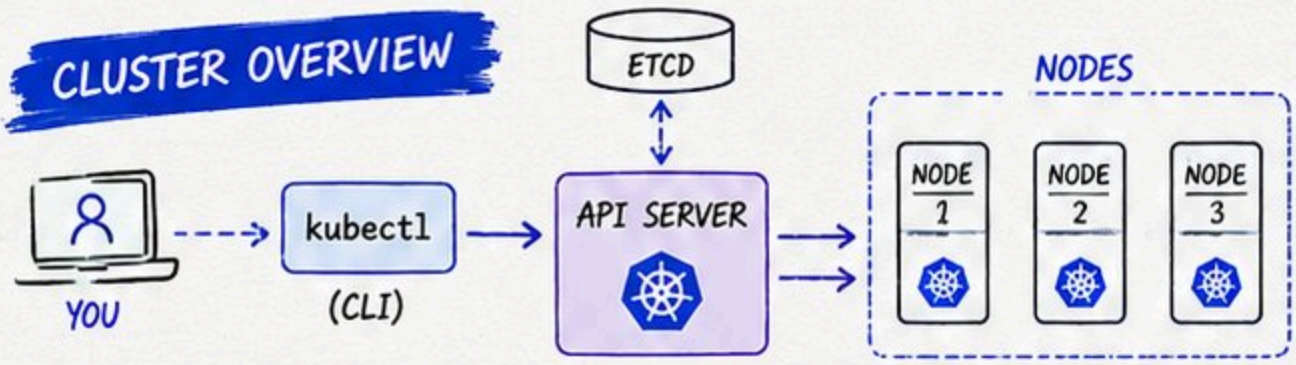
- Shows all worker and control-plane nodes.
- Check node health at a glance.

## 5 NODE METRICS

```
$ kubectl top nodes
NAME      CPU(cores)   CPU%   MEMORY(bytes)   MEMORY%
node-01   250m         12%    1534Mi          35%
node-02   180m         9%     1024Mi          28%
node-03   320m         16%    2048Mi          42%
```

- Displays resource usage (CPU/Memory) for nodes.
- Requires metrics-server installed.

## CLUSTER OVERVIEW




kubectl talks to the API Server, which manages the state of your entire cluster.

## PRO TIP

Always run these first:

- ☑ kubectl config get-contexts
- ☑ kubectl config use-context <name>
- ☑ kubectl cluster-info

It saves time, prevents mistakes, and keeps your cluster healthy!

★ Know your context. Know your cluster. Deploy with confidence.

KUBERNETES CHEAT SHEET

# POD COMMANDS

Everything starts (and often ends) with your Pods.

Pods are the smallest deployable units in Kubernetes.

## 1 LIST PODS

```
$ kubectl get pods
NAME                     READY   STATUS    RESTARTS   AGE
nginx-7d7f9c4b8-abc12    1/1     Running   0          2d
api-5d5f7c8d9-xyz34      1/1     Running   1          1d
web-6f6g8h9i0-jkl56      1/1     Running   0          3h
```

List all pods in the current namespace.

## 2 INSPECT POD

```
$ kubectl describe pod <pod-name>
# Shows detailed info about the pod,
# including events, conditions, IP,
# mounts, and more.
```

Deep dive into a pod's configuration and current state.

## 3 VIEW LOGS

```
$ kubectl logs <pod-name>
# Add -f to follow logs in real-time
$ kubectl logs -f <pod-name>
```

View logs from a pod to troubleshoot issues quickly.

## 4 EXEC INTO POD

```
$ kubectl exec -it <pod-name> -- bash
# Open an interactive shell
# inside the running container.
```

Access a running container in a pod for debugging or inspection.

## 5 DELETE POD

```
$ kubectl delete pod <pod-name>
# Deletes the specified pod.
# It will be recreated if managed
# by a controller (e.g., Deployment).
```

Remove a pod. It may be recreated by controllers.

## PRO TIP

Most Kubernetes troubleshooting starts with these commands.

## POD TROUBLESHOOTING FLOW

1. Get Pods — See what's running
2. Describe — Understand the details
3. Logs — Find errors in real-time
4. Exec — Debug inside the container
5. Fix & Verify — Apply fix and recheck

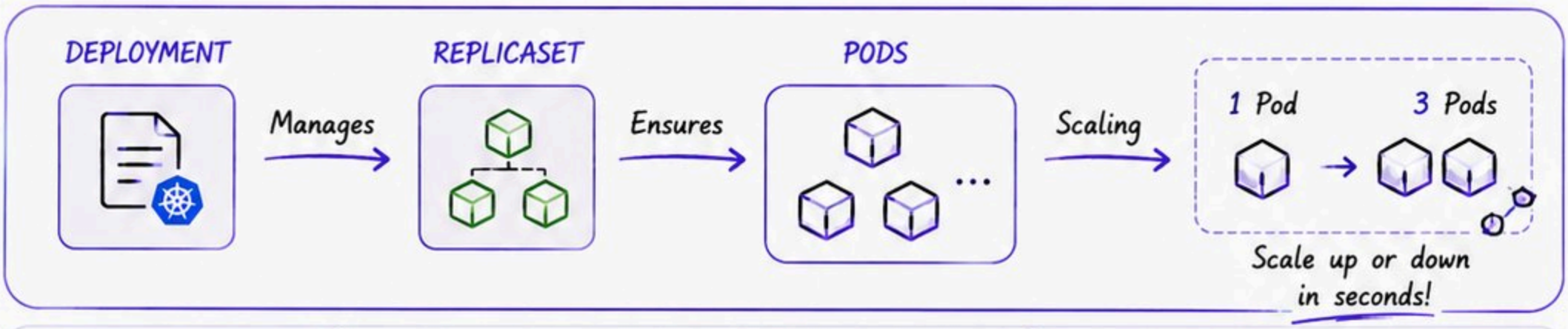
Happy Debugging!

KUBERNETES CHEAT SHEET

# DEPLOYMENTS & SCALING

Deployments manage ReplicaSets and keep your app in the desired state.

Deploy, update and scale your applications with confidence.




## 1 LIST DEPLOYMENTS

```
$ kubectl get deployments
NAME    READY   UP-TO-DATE   AVAILABLE   AGE
app     3/3     3            3           2d
web     2/2     2            2           1d
```

- List all deployments in the current namespace.

## 2 APPLY DEPLOYMENT

```
$ kubectl apply -f deploy.yaml
# Creates or updates the deployment
# defined in deploy.yaml
```

- Create or update a deployment from a YAML file.

## 3 CHECK ROLLOUT STATUS

```
$ kubectl rollout status deployment/app
# Shows the rollout progress and
# whether the deployment is successful
```

- Monitor the rollout progress.
- Ensures your app is updated correctly.

## 4 SCALE DEPLOYMENT

```
$ kubectl scale deployment app --replicas=3
deployment.apps/app scaled
```

- Scale your application up or down instantly.
- Adjust replicas as per demand.

## 5 ROLLBACK DEPLOYMENT

```
$ kubectl rollout undo deployment/app
# Rollback to the previous
# stable version
```

- Undo a faulty rollout.
- Quick recovery = less downtime.

### PRO TIP

Always check rollout status after applying changes. It saves debugging time!

### USEFUL ADD-ONS

```
$ kubectl rollout history deployment/app
# View rollout history

$ kubectl rollout history deployment/app --revision=2
# View details of a specific revision

$ kubectl rollout undo deployment/app --to-revision=2
# Rollback to a specific revision
```

Rollback is not a failure, it's a superpower!

KUBERNETES CHEAT SHEET

# SERVICES & NETWORKING

Expose, connect and communicate your applications.

Services give your Pods a stable identity and network access.

## TRAFFIC FLOW


### 1 LIST SERVICES

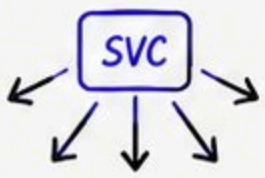
```
$ kubectl get svc
NAME          TYPE        CLUSTER-IP      PORT(S)   AGE
kubernetes    ClusterIP   10.96.0.1       443/TCP   10d
web-service   ClusterIP   10.96.123.45    80/TCP    2d
```

- List all services in the current namespace.

### 2 EXPOSE DEPLOYMENT

```
$ kubectl expose deployment app \
  --port=80 --target-port=8080 \
  --type=ClusterIP --name=app-service
service/app-service exposed
```

- Expose a deployment as a service.
- Creates a stable endpoint.




### 3 LIST INGRESS

```
$ kubectl get ingress
NAME          CLASS   HOSTS             ADDRESS       PORTS   AGE
web-ingress   nginx   app.example.com   34.101.20.1   80,443  3d
```

- List ingress resources for HTTP/HTTPS routing.


### 4 PORT-FORWARD

```
$ kubectl port-forward pod/app 8080:80
Forwarding from 127.0.0.1:8080 -> 80
```

- Access a pod or service locally.
- Great for debugging and testing.


### 5 VIEW ENDPOINTS

```
$ kubectl get endpoints app-service
NAME          ENDPOINTS                          AGE
app-service   10.244.1.5:8080,10.244.2.7:8080    2d
```

- See the IPs and ports backing a service.
- Shows where traffic is routed.

## SERVICE TYPES QUICK VIEW

| | |
|---|---|
| ClusterIP | Exposes the service on a cluster-internal IP. (Default) |
| NodePort | Exposes the service on each Node's IP at a static port. |
| LoadBalancer | Exposes the service externally using a cloud provider's load balancer. |
| ExternalName | Maps the service to an external DNS name. (No proxying) |

## PRO TIP

- [x] Use ClusterIP for internal communication.
- [x] Use Ingress for HTTP/HTTPS access.
- [x] Use port-forward for local debugging.
- [x] Always check Endpoints when traffic is not reaching Pods.

Good networking makes your app FAST, RELIABLE & SECURE!

Save this slide for your next debugging session!

KUBERNETES CHEAT SHEET

# NAMESPACES & RESOURCE CONTROL

Organize, isolate and control your cluster resources.

Namespaces help you separate workloads.

## MULTI-ENVIRONMENT CLUSTER SETUP


Isolation • Organization • Security • Resource Management

### 1 LIST NAMESPACES

```
$ kubectl get ns
NAME              STATUS   AGE
default           Active   10d
kube-system       Active   10d
dev               Active   5d
qa                Active   5d
prod              Active   5d
```

List all namespaces in the cluster.

### 2 CREATE NAMESPACE

```
$ kubectl create ns dev
namespace/dev created
```

Create a new namespace.


### 3 SET CURRENT NAMESPACE

```
$ kubectl config set-context --current
  --namespace=dev
Context "minikube" modified.
```

Switch your current context to the desired namespace.

### 4 LIST RESOURCE QUOTAS

```
$ kubectl get quota -n dev
NAME              AGE
dev-quota         5d
```

View resource quotas applied in a namespace.

### 5 DESCRIBE QUOTA

```
$ kubectl describe quota dev-quota -n dev
Name:       dev-quota
Namespace:  dev
Resource    Used    Hard
--------    ----    ----
cpu         2       4
memory      4Gi     8Gi
pods        10      20
```

Check details of a quota: limits and current usage.

## WHY NAMESPACES?

- Isolate environments (Dev, QA, Prod)
- Organize workloads by teams or projects
- Apply resource quotas and limits
- Improve security and access control
- Avoid naming conflicts

## PRO TIP

Always work in the right namespace! Accidentally deploying in the wrong one can cause serious issues.

```
$ kubectl config view --minify | grep namespace
```

Check your active namespace

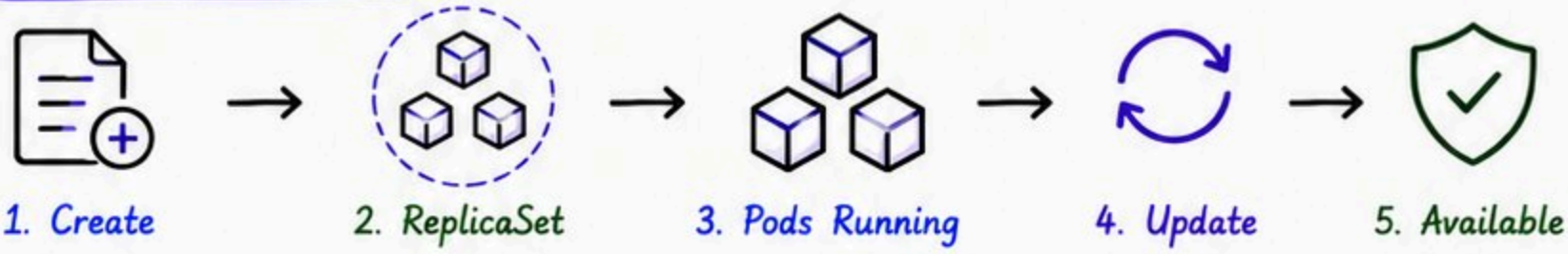
KUBERNETES CHEAT SHEET

# DEPLOYMENTS & PODS MANAGEMENT

Deployments ensure your apps are always available and resilient.

Deploy, update and manage your applications with ease.

## DEPLOYMENT LIFECYCLE




Kubermetes ensures the desired state is always maintained.

## COMMON DEPLOYMENT COMMANDS

| # | COMMAND | DESCRIPTION | EXAMPLE |
|---|---|---|---|
| 1 | `kubectl create deployment <name> --image=<image>` | Create a new deployment. | `kubectl create deployment nginx --image=nginx:1.25` |
| 2 | `kubectl get deployments` | List all deployments. | `kubectl get deployments` |
| 3 | `kubectl describe deployment <name>` | Show details of a deployment. | `kubectl describe deploy nginx` |
| 4 | `kubectl scale deployment <name> --replicas=<n>` | Scale the deployment. | `kubectl scale deploy nginx --replicas=5` |
| 5 | `kubectl set image deployment/<name> <container>=<image>` | Update container image of a deployment. | `kubectl set image deploy nginx nginx=nginx:1.26` |
| 6 | `kubectl rollout status deployment/<name>` | Check rollout status. | `kubectl rollout status deploy/nginx` |
| 7 | `kubectl rollout undo deployment/<name>` | Rollback to previous revision. | `kubectl rollout undo deploy/nginx` |
| 8 | `kubectl delete deployment <name>` | Delete a deployment. | `kubectl delete deploy nginx` |

## PODS MANAGEMENT

| COMMAND | DESCRIPTION | EXAMPLE |
|---|---|---|
| `kubectl get pods` | List all pods in the current namespace. | `kubectl get pods` |
| `kubectl describe pod <name>` | Show details of a pod. | `kubectl describe pod nginx-pod` |
| `kubectl logs <pod-name>` | View logs of a pod. | `kubectl logs nginx-pod` |
| `kubectl exec -it <pod-name> -- /bin/sh` | Execute a command inside a pod. | `kubectl exec -it nginx-pod -- /bin/sh` |
| `kubectl delete pod <name>` | Delete a pod. | `kubectl delete pod nginx-pod` |

## BEST PRACTICES

- Use Deployments for stateless applications.
- Always set resource requests and limits.
- Use liveness and readiness probes.
- Keep images small and up to date.
- Monitor rollouts and use rollbacks if needed.

## PRO TIP

Use rollout history to track changes and revert if something goes wrong.

```
kubectl rollout history deploy/<name>
```

Example:

```
kubectl rollout history deploy/nginx
```

Automate. Monitor. Scale. Repeat.
KUBERNETES MAKES IT SIMPLE!

Save this slide for your next debugging session!

KUBERNETES CHEAT SHEET

# CONFIGMAPS & SECRETS

Store and manage configuration data and sensitive information securely.

Use ConfigMaps for non-sensitive data and Secrets for sensitive data.

## CONFIGMAP

- Store non-sensitive configuration data in key-value pairs.
- Used for app settings, feature flags, file content, etc.

## SECRET

- Store sensitive data like passwords, tokens, keys, etc.
- Data is base64 encoded (not encrypted by default).

## HOW THEY WORK IN PODS

ConfigMap / Secret (or) → Mounted as Volume → Pod → Injected as Environment Variables → KEY=value KEY=value ...

Choose the right method based on your application needs.

## CONFIGMAP COMMANDS

| # | COMMAND | DESCRIPTION | EXAMPLE |
|---|---|---|---|
| 1 | `kubectl create configmap <name> --from-literal=key=value` | Create ConfigMap from literal values. | `kubectl create configmap app-config \ --from-literal=APP_MODE=prod` |
| 2 | `kubectl create configmap <name> --from-file=<file>` | Create ConfigMap from a file. | `kubectl create configmap app-config \ --from-file=config.properties` |
| 3 | `kubectl get configmaps` | List all ConfigMaps in the namespace. | `kubectl get configmaps` |
| 4 | `kubectl describe configmap <name>` | Show details of a ConfigMap. | `kubectl describe configmap <name>` |
| 5 | `kubectl delete configmap <name>` | Delete a ConfigMap. | `kubectl delete configmap <name>` |

## SECRET COMMANDS

| # | COMMAND | DESCRIPTION | EXAMPLE |
|---|---|---|---|
| 1 | `kubectl create secret generic <name> --from-literal=key=value` | Create a generic Secret from literals. | `kubectl create secret generic db-secret \ --from-literal=DB_USER=admin \ --from-literal=DB_PASS=myPass123` |
| 2 | `kubectl create secret generic <name> --from-file=<file>` | Create a Secret from a file. | `kubectl create secret generic tls-secret \ --from-file=tls.crt --from-file=tls.key` |
| 3 | `kubectl get secrets` | List all Secrets in the namespace. | `kubectl get secrets` |
| 4 | `kubectl describe secret <name>` | Show details of a Secret. | `kubectl describe secret db-secret` |
| 5 | `kubectl delete secret <name>` | Delete a Secret. | `kubectl delete secret db-secret` |

## USING CONFIGMAPS & SECRETS IN PODS

### 1. AS VOLUME (FILES)

```
volumes:
  - name: config-volume
    configMap:
      name: app-config
volumeMounts:
  - name: config-volume
    mountPath: /etc/config
```

Files will be available in /etc/config

### 2. AS ENVIRONMENT VARIABLES

```
env:
  - name: APP_MODE
    valueFrom:
      configMapKeyRef:
        name: app-config
        key: APP_MODE
```

Environment variable available to the container

## BEST PRACTICES

- Use ConfigMaps for non-sensitive configuration.
- Use Secrets for sensitive data.
- Enable RBAC to restrict access to Secrets.
- Avoid storing sensitive data in ConfigMaps.
- Rotate Secrets regularly.

## PRO TIP

Consider using external secret management tools like HashiCorp Vault, AWS Secrets Manager, or Azure Key Vault for enhanced security.

```
kubectl get secret my-secret -o yaml
```

↳ Check your Secrets safely

Secure your configs. Protect your secrets.
**BUILD TRUST, DEPLOY CONFIDENTLY!**

Save this slide for your next debugging session!

KUBERNETES CHEAT SHEET

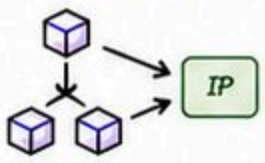
# SERVICES & NETWORKING

*Expose your applications and enable communication between services.*

Kubernetes provides powerful networking primitives to connect your applications inside and outside the cluster.

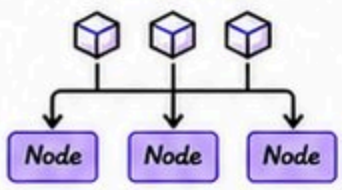
## SERVICE TYPES

### ClusterIP



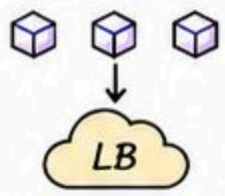

*Exposes the Service on a cluster-internal IP. Accessible only within the cluster.*

### NodePort




*Exposes the Service on each Node's IP at a static port (NodePort). Accessible from outside using <NodeIP>:<Port>.*

### LoadBalancer

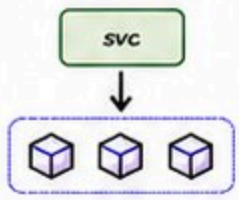



*Exposes the Service externally using a Cloud Provider's Load Balancer. Accessible via an external IP/DNS.*

### ExternalName


*Maps the Service to the contents of the external Name field (CNAME record).*

### Headless




*Does not allocate a ClusterIP. Used for service discovery (e.g., StatefulSets).*

## COMMON SERVICE COMMANDS

| # | COMMAND | DESCRIPTION | EXAMPLE |
|---|---|---|---|
| 1 | `kubectl get svc` | *List all Services in the current namespace.* | `kubectl get svc` |
| 2 | `kubectl describe svc <name>` | *Show details of a Service.* | `kubectl describe svc nginx` |
| 3 | `kubectl expose deployment <name> \ --port=<port> --target-port=<port>` | *Expose a Deployment as a Service.* | `kubectl expose deploy nginx \ --port=80 --target-port=8080` |
| 4 | `kubectl get endpoints <name>` | *Show endpoints (Pods IPs) behind a Service.* | `kubectl get endpoints nginx` |
| 5 | `kubectl delete svc <name>` | *Delete a Service.* | `kubectl delete svc nginx` |

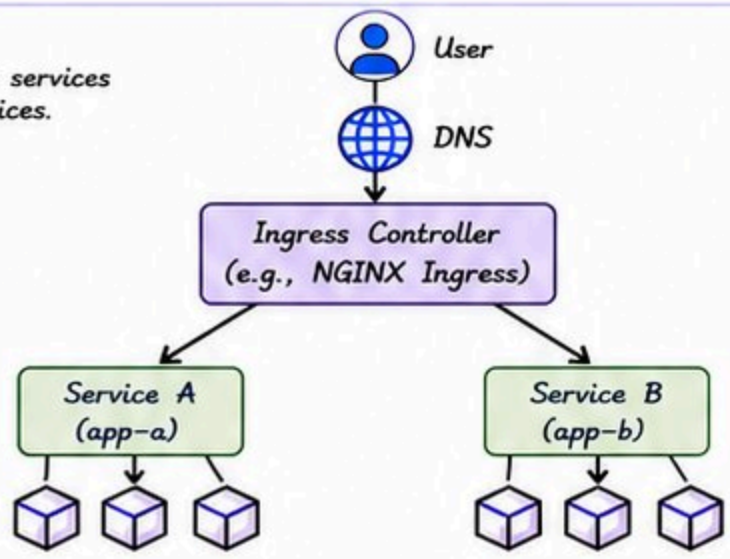
## INGRESS

*Ingress manages external access to services in a more powerful way than Services. It provides:*

- *Host-based routing*
- *Path-based routing*
- *TLS/SSL termination*
- *Load balancing*

**DID YOU KNOW?**

*Ingress Controller is not part of Kubernetes core. You need to install one (e.g., NGINX, Traefik, HAProxy, etc.).*




**Example Ingress (YAML)**

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: my-ingress
spec:
 rules:
 - host: myapp.com
   http:
     paths:
     - path: /app-a
       pathType: Prefix
       backend:
         service:
           name: service-a
           port:
             number: 80
     - path: /app-b
       pathType: Prefix
       backend:
         service:
           name: service-b
           port:
             number: 80
```

## DNS IN KUBERNETES

*Kubernetes has built-in DNS (CoreDNS) that provides DNS records for Services.*

DNS Format

`<service-name>.<namespace>.svc.cluster.local`

(e.g., my-svc.default.svc.cluster.local)

**Common DNS Queries**

| | | | |
|---|---|---|---|
| ○ | my-svc.default.svc.cluster.local | → | Service (ClusterIP) |
| ○ | my-svc.default.svc.cluster.local. | → | Service (FQDN) |
| ○ | pod-ip-address | → | Pod |
| ○ | my-headless.default.svc.cluster.local | → | All Pod IPs (Headless Service) |

## BEST PRACTICES

- *Use ClusterIP for internal communication.*
- *Use NodePort for testing or bare metal setups.*
- *Use LoadBalancer for external access in cloud.*
- *Prefer Ingress over multiple LoadBalancers.*
- *Use meaningful names for Services and Ingress rules.*

## PRO TIP

*Use labels and selectors consistently to keep your Services decoupled from Pod IPs.*

```
kubectl get svc --show-labels
```

*Good networking makes great applications!*

**CONNECT. ROUTE. SCALE.**

*Save this slide for your next debugging session!*

KUBERNETES CHEAT SHEET

# KUBERNETES CHEAT SHEET

Quick reference for managing your Kubernetes cluster like a pro!

Kubernetes helps you run, scale, and manage containerized applications reliably.

## RESOURCE OVERVIEW

| Resource | Purpose |
|---|---|
| Pod | Smallest deployable unit. Holds one or more containers. |
| Deployment | Manages ReplicaSets and ensures desired state. |
| ReplicaSet | Maintains a stable set of replica Pods. |
| Service | Exposes Pods to other services or outside. |
| Ingress | Manages external access (HTTP/HTTPS) to services. |
| ConfigMap | Stores non-sensitive configuration data. |
| Secret | Stores sensitive data like passwords and tokens. |
| PersistentVolume | Provides storage in the cluster. |
| Namespace | Logical isolation within a cluster. |

## USEFUL kubectl COMMANDS

| Category | Command | Description | Example |
|---|---|---|---|
| Cluster Info | `kubectl cluster-info` | Display cluster information | `kubectl cluster-info` |
| | `kubectl get nodes` | List all nodes | `kubectl get nodes` |
| | `kubectl get pods -A` | List all pods in all namespaces | `kubectl get pods -A` |
| Workloads | `kubectl get deploy` | List all deployments | `kubectl get deploy` |
| | `kubectl get rs` | List all ReplicaSets | `kubectl get rs` |
| | `kubectl get pods` | List pods in current namespace | `kubectl get pods` |
| | `kubectl logs <pod>` | View logs from a pod | `kubectl logs my-pod` |
| | `kubectl exec -it <pod> -- sh` | Open shell in a pod | `kubectl exec -it my-pod -- sh` |
| | `kubectl describe pod <pod>` | Show detailed info about a pod | `kubectl describe pod my-pod` |
| Services & Network | `kubectl get svc` | List all services | `kubectl get svc` |
| | `kubectl describe svc <svc>` | Show details of a service | `kubectl describe svc my-svc` |
| | `kubectl get ingress` | List all ingress resources | `kubectl get ingress` |
| | `kubectl describe ingress <ing>` | Show details of an ingress | `kubectl describe ingress my-ing` |
| Config & Storage | `kubectl get configmap` | List all ConfigMaps | `kubectl get configmap` |
| | `kubectl get secret` | List all Secrets | `kubectl get secret` |
| | `kubectl get pvc` | List all PersistentVolumeClaims | `kubectl get pvc` |
| | `kubectl get pv` | List all PersistentVolumes | `kubectl get pv` |
| Management | `kubectl apply -f <file.yaml>` | Apply configuration from file | `kubectl apply -f app.yaml` |
| | `kubectl delete -f <file.yaml>` | Delete resources from file | `kubectl delete -f app.yaml` |
| | `kubectl rollout status deploy/<name>` | Check rollout status | `kubectl rollout status deploy/my-app` |
| | `kubectl scale deploy/<name> --replicas=3` | Scale a deployment | `kubectl scale deploy/my-app --replicas=3` |

## POD LIFE CYCLE

**Pending** — Pod is accepted by the cluster but not yet scheduled.

**Running** — Pod is bound to a node and containers are running.

**Succeeded** — Pod completed successfully and will not be restarted.

**Failed** — Pod failed in execution and will not be restarted.

**Unknown** — Pod state can't be determined (e.g., node unreachable).

## YAML TIPS

- Always use apiVersion, kind, metadata, and spec.
- Use meaningful names for resources.
- Indentation matters! YAML uses spaces, not tabs.
- Validate your YAML:

```
kubectl apply -f file.yaml --dry-run=client
```

- Use comments to document your YAML files.

## NAMESPACE SHORTCUTS

| Command | Description |
|---|---|
| `kubectl get ns` | List all namespaces |
| `kubectl get all -n <ns>` | List all resources in a namespace |
| `kubectl delete ns <ns>` | Delete a namespace |
| `kubectl config set-context --current --namespace=<ns>` | Switch to a namespace |

## DEBUGGING TIPS

- Check pod status: `kubectl get pods`
- Describe a pod: `kubectl describe pod <pod>`
- View logs: `kubectl logs <pod>`
- Check events: `kubectl get events --sort-by=.metadata.creationTimestamp`
- Exec into a pod: `kubectl exec -it <pod> -- sh`

## BEST PRACTICES

- Use namespaces to organize resources.
- Use labels and selectors consistently.
- Set resource requests and limits for containers.
- Use Helm charts for managing complex applications.
- Keep your images small and secure.
- Regularly backup etcd and critical data.

## PRO TIP

Automate deployments, monitor your cluster, and keep learning!

```
kubectl get pods -A --watch
```

Kubernetes is powerful in your hands!

BUILD. DEPLOY. SCALE. REPEAT.

Save this slide for your next debugging session!